

```
# =====  
# COMPLETE SINGLE-CELL PROJECT  
# =====  
  
import os  
import zipfile  
import warnings  
warnings.filterwarnings('ignore')  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.metrics import (  
    confusion_matrix,  
    classification_report  
)  
  
import tensorflow as tf  
  
from tensorflow.keras.preprocessing.image import (  
    ImageDataGenerator,  
    load_img,  
    img_to_array  
)  
  
from tensorflow.keras.models import Model, Sequential  
  
from tensorflow.keras.layers import (  
    Dense,  
    Dropout,  
    Flatten,  
    Conv2D,  
    MaxPooling2D,  
    GlobalAveragePooling2D  
)  
  
from tensorflow.keras.applications import (  
    VGG16,  
    ResNet50,  
    DenseNet121  
)
```

```

# =====
# STEP 1 : EXTRACT ZIP FILE
# =====

ZIP_FILE = "archive (3).zip"

EXTRACT_PATH = "./extracted_dataset/"

with zipfile.ZipFile(ZIP_FILE, 'r') as zip_ref:
    zip_ref.extractall(EXTRACT_PATH)

print("Dataset Extracted Successfully")

# =====
# STEP 2 : FIND DATASET FOLDER
# =====

dataset_path = None

for root, dirs, files in os.walk(EXTRACT_PATH):
    if len(dirs) >= 2:
        dataset_path = root
        break

print("Dataset Path:", dataset_path)

# =====d
# STEP 3 : IMAGE GENERATOR
# =====

IMG_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 5

# Create separate data generators for training and testing
train_dir = os.path.join(dataset_path, 'train')
test_dir = os.path.join(dataset_path, 'test')

# Check if the directories exist (optional, but good for debugging)
if not os.path.exists(train_dir):
    print(f"Error: Training directory not found at {train_dir}")
if not os.path.exists(test_dir):
    print(f"Error: Testing directory not found at {test_dir}")

```

```

train_datagen = ImageDataGenerator(
    rescale=1./255
)

val_datagen = ImageDataGenerator(
    rescale=1./255
)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)

val_generator = val_datagen.flow_from_directory(
    test_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

NUM_CLASSES = len(train_generator.class_indices)

print("\nClasses:")
print(train_generator.class_indices)

# =====
# STEP 4 : CNN MODEL
# =====

def CNN_Model():
    model = Sequential([
        Conv2D(
            32,
            (3,3),
            activation='relu',
            input_shape=(224,224,3)
        ),
        MaxPooling2D(),

```

```

        Conv2D(
            64,
            (3,3),
            activation='relu'
        ),
        MaxPooling2D(),
        Conv2D(
            128,
            (3,3),
            activation='relu'
        ),
        MaxPooling2D(),
        Flatten(),
        Dense(
            256,
            activation='relu'
        ),
        Dropout(0.5),
        Dense(
            NUM_CLASSES,
            activation='softmax'
        )
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

```

```

# =====
# STEP 5 : VGG16
# =====

```

```
def VGG16_Model():
```

```

base = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

base.trainable = False

x = GlobalAveragePooling2D()(base.output)

x = Dense(
    256,
    activation='relu'
)(x)

x = Dropout(0.5)(x)

output = Dense(
    NUM_CLASSES,
    activation='softmax'
)(x)

model = Model(
    base.input,
    output
)

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

return model

# =====
# STEP 6 : RESNET50
# =====

def ResNet_Model():

    base = ResNet50(
        weights='imagenet',
        include_top=False,
        input_shape=(224,224,3)
    )

```

```

    )

    base.trainable = False

    x = GlobalAveragePooling2D()(base.output)

    x = Dense(
        256,
        activation='relu'
    )(x)

    x = Dropout(0.5)(x)

    output = Dense(
        NUM_CLASSES,
        activation='softmax'
    )(x)

    model = Model(
        base.input,
        output
    )

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

# =====
# STEP 7 : DENSENET121
# =====

def DenseNet_Model():

    base = DenseNet121(
        weights='imagenet',
        include_top=False,
        input_shape=(224,224,3)
    )

    base.trainable = False

```

```

x = GlobalAveragePooling2D()(base.output)

x = Dense(
    256,
    activation='relu'
)(x)

x = Dropout(0.5)(x)

output = Dense(
    NUM_CLASSES,
    activation='softmax'
)(x)

model = Model(
    base.input,
    output
)

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

return model

# =====
# STEP 8 : TRAIN ALL MODELS
# =====

models = {
    "CNN": CNN_Model(),
    "VGG16": VGG16_Model(),
    "ResNet50": ResNet_Model(),
    "DenseNet121": DenseNet_Model()
}

histories = {}
accuracies = {}

for name, model in models.items():

    print("\n")
    print("="*50)

```

```

print("Training:", name)
print("="*50)

history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=EPOCHS,
    verbose=1
)

loss, acc = model.evaluate(
    val_generator,
    verbose=0
)

histories[name] = history
accuracies[name] = acc

print(name, "Accuracy:", round(acc*100,2), "%")

# =====
# STEP 9 : ACCURACY COMPARISON
# =====

plt.figure(figsize=(8,5))

plt.bar(
    accuracies.keys(),
    accuracies.values()
)

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")

plt.show()

# =====
# STEP 10 : TRAINING CURVES
# =====

for name, history in histories.items():

    plt.figure(figsize=(8,5))

    plt.plot(

```



```

        history.history['accuracy'],
        label='Train Accuracy'
    )

    plt.plot(
        history.history['val_accuracy'],
        label='Validation Accuracy'
    )

    plt.title(name)

    plt.xlabel("Epoch")

    plt.ylabel("Accuracy")

    plt.legend()

    plt.show()

# =====
# STEP 11 : BEST MODEL
# =====

best_model_name = max(
    accuracies,
    key=accuracies.get
)

best_model = models[
    best_model_name
]

print("\nBest Model:", best_model_name)

# =====
# STEP 12 : CONFUSION MATRIX
# =====

predictions = best_model.predict(
    val_generator
)

y_pred = np.argmax(
    predictions,
    axis=1

```

```
)

y_true = val_generator.classes

cm = confusion_matrix(
    y_true,
    y_pred
)

plt.figure(figsize=(8,6))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title(
    f'Confusion Matrix ({best_model_name})'
)

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.show()

# =====
# STEP 13 : CLASSIFICATION REPORT
# =====

print(
    classification_report(
        y_true,
        y_pred,
        target_names=list(
            train_generator.class_indices.keys()
        )
    )
)
```


Dataset Extracted Successfully
Dataset Path: ./extracted_dataset/Covid19-dataset
Found 251 images belonging to 3 classes.
Found 66 images belonging to 3 classes.

Classes:
{'Covid': 0, 'Normal': 1, 'Viral Pneumonia': 2}

=====
Training: CNN
=====

Epoch 1/5
8/8  12s 1s/step - accuracy: 0.4343 - loss: 1.6515 - val_accuracy: 0.3788 - \nEpoch 2/5
8/8  6s 758ms/step - accuracy: 0.7092 - loss: 0.6238 - val_accuracy: 0.7879 - \nEpoch 3/5
8/8  6s 666ms/step - accuracy: 0.8685 - loss: 0.3349 - val_accuracy: 0.7879 - \nEpoch 4/5
8/8  6s 811ms/step - accuracy: 0.8924 - loss: 0.2912 - val_accuracy: 0.8333 - \nEpoch 5/5
8/8  6s 720ms/step - accuracy: 0.9084 - loss: 0.2169 - val_accuracy: 0.8182 - \nCNN Accuracy: 81.82 %

=====
Training: VGG16
=====

Epoch 1/5
8/8  14s 1s/step - accuracy: 0.3944 - loss: 1.1889 - val_accuracy: 0.6364 - \nEpoch 2/5
8/8  6s 790ms/step - accuracy: 0.6056 - loss: 0.8831 - val_accuracy: 0.6818 - \nEpoch 3/5
8/8  7s 886ms/step - accuracy: 0.6414 - loss: 0.8322 - val_accuracy: 0.8030 - \nEpoch 4/5
8/8  6s 783ms/step - accuracy: 0.6853 - loss: 0.7112 - val_accuracy: 0.7121 - \nEpoch 5/5
8/8  7s 896ms/step - accuracy: 0.7888 - loss: 0.6200 - val_accuracy: 0.8485 - \nVGG16 Accuracy: 84.85 %

=====
Training: ResNet50

Training: ResNet50

=====

Epoch 1/5

8/8  35s 3s/step - accuracy: 0.3745 - loss: 1.4286 - val_accuracy: 0.4242 - \

Epoch 2/5

8/8  6s 738ms/step - accuracy: 0.4303 - loss: 1.1929 - val_accuracy: 0.4242 -

Epoch 3/5

8/8  7s 838ms/step - accuracy: 0.4940 - loss: 1.0401 - val_accuracy: 0.8030 -

Epoch 4/5

8/8  6s 742ms/step - accuracy: 0.5458 - loss: 0.9596 - val_accuracy: 0.5909 -

Epoch 5/5

8/8  7s 860ms/step - accuracy: 0.5737 - loss: 0.9135 - val_accuracy: 0.8030 -

ResNet50 Accuracy: 80.3 %

=====

Training: DenseNet121

=====

Epoch 1/5

8/8  77s 6s/step - accuracy: 0.4980 - loss: 1.4752 - val_accuracy: 0.4697 - \

Epoch 2/5

8/8  7s 849ms/step - accuracy: 0.6813 - loss: 0.6957 - val_accuracy: 0.7121 -

Epoch 3/5

8/8  6s 723ms/step - accuracy: 0.8406 - loss: 0.4016 - val_accuracy: 0.7121 -

Epoch 4/5

8/8  7s 842ms/step - accuracy: 0.9044 - loss: 0.3097 - val_accuracy: 0.8788 -

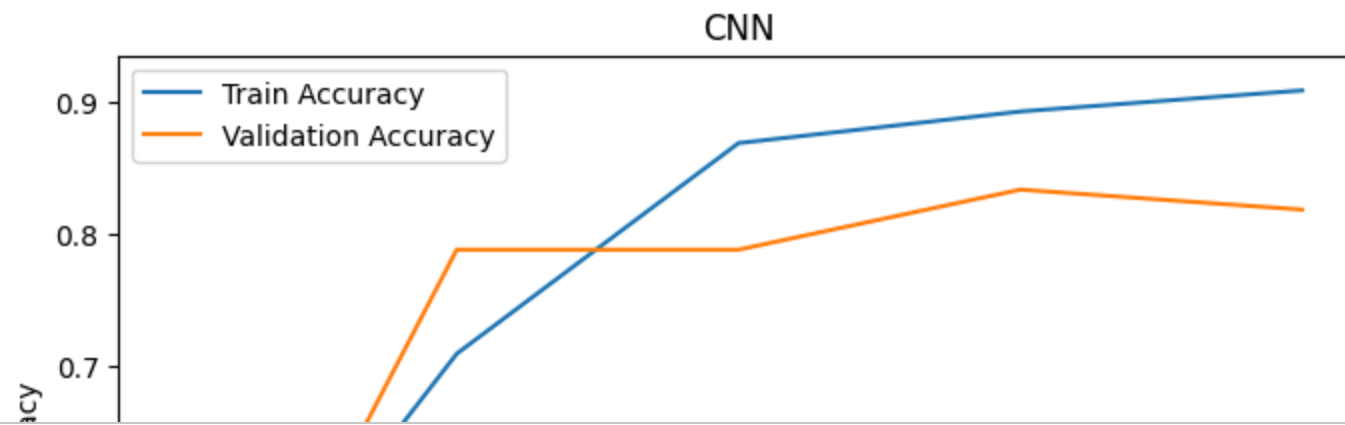
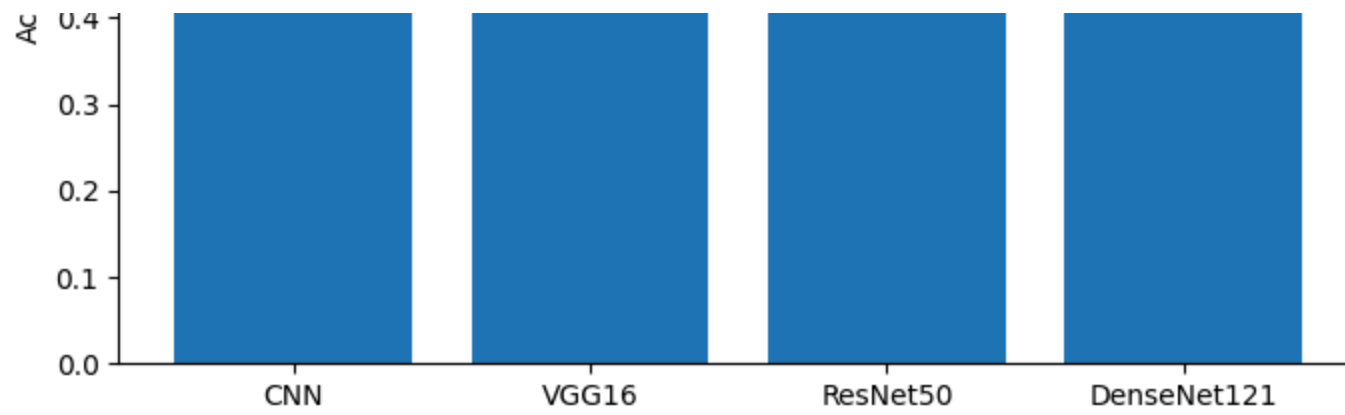
Epoch 5/5

8/8  6s 719ms/step - accuracy: 0.9084 - loss: 0.2601 - val_accuracy: 0.8485 -

DenseNet121 Accuracy: 84.85 %

Model Accuracy Comparison





```

# =====
# STEP 14 : PREDICT NEW IMAGE (SEPARATE CELL)
# =====

import os
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import (
    load_img,
    img_to_array
)

# Enter test image path
IMAGE_PATH = "s5.jpg" # Change image name if needed

if not os.path.exists(IMAGE_PATH):
    print(f"Error: Image not found -> {IMAGE_PATH}")
else:
    # Load image
    img = load_img(
        IMAGE_PATH,
        target_size=(224, 224)
    )

    # Display image
    plt.figure(figsize=(5,5))
    plt.imshow(img)
    plt.axis("off")
    plt.title("Input X-Ray")
    plt.show()

    # Preprocess image
    img_array = img_to_array(img)

    img_array = img_array.astype("float32") / 255.0

    img_array = np.expand_dims(
        img_array,
        axis=0
    )

    # Prediction

```

```

pred = best_model.predict(
    img_array,
    verbose=0
)

# Class names
classes = list(
    train_generator.class_indices.keys()
)

# Probabilities
print("\nPrediction Probabilities:")
print("-"*40)

for i, cls in enumerate(classes):
    print(
        f"{cls:<20}: {pred[0][i]*100:.2f}%"
    )

# Best prediction
max_prob = np.max(pred)

predicted_index = np.argmax(pred)

disease = classes[predicted_index]

confidence = max_prob * 100

# Confidence threshold
if confidence < 70:
    disease = "Uncertain"

print("\n" + "="*40)
print("FINAL PREDICTION")
print("="*40)

print(
    f"Predicted Disease : {disease}"
)

print(
    f"Confidence Score : {confidence:.2f}%"
)

```


Input X-Ray



Prediction Probabilities:

```
-----  
Covid           : 51.71%  
Normal          : 37.44%  
Viral Pneumonia : 10.85%
```

```
# =====  
# COMPLETE AI MEDICAL REPORT (SINGLE CELL)  
# =====  
  
# Install reportlab if not already installed  
!pip install reportlab  
  
import pandas as pd  
import zipfile # Import zipfile module  
import os # Import os module to check for file existence  
from reportlab.platypus import (
```

```

SimpleDocTemplate,
Paragraph,
Spacer,
PageBreak
)
from reportlab.lib.styles import getSampleStyleSheet

# -----
# LOAD KAGGLE MEDICAL DATABASE
# -----

# Define the path to the medical dataset zip file
medical_zip_path = "/content/archive (4).zip"

# Check if the zip file exists before attempting to extract
if os.path.exists(medical_zip_path):
    # Extract the medical dataset zip file
    with zipfile.ZipFile(
        medical_zip_path,
        'r'
    ) as zip_ref:
        zip_ref.extractall("./medical_dataset")

medical_df = pd.read_csv(
    "./medical_dataset/Final_Augmented_dataset_Diseases_and_Symptoms.csv" # Corrected path
)

symptom_columns = medical_df.columns[1:]

# -----
# MAP X-RAY CLASSES TO DISEASE DATASET
# -----

disease_mapping = {
    "Covid": "influenza",
    "COVID": "influenza",
    "Viral Pneumonia": "pneumonia",
    "Normal": None
}

mapped_disease = disease_mapping.get(
    disease,
    disease.lower()
)

```

```

# -----
# EXTRACT SYMPTOMS
# -----

def get_symptoms(mapped_name):

    if mapped_name is None:
        return ["No symptoms detected (Healthy Condition)"]

    disease_rows = medical_df[
        medical_df['diseases'].astype(str).str.lower()
        == mapped_name.lower()
    ]

    if len(disease_rows) == 0:
        return ["No symptom information available"]

    first_row = disease_rows.iloc[0]

    symptoms = []

    for col in symptom_columns:

        try:
            if first_row[col] == 1:
                symptoms.append(col)
        except:
            pass

    return symptoms[:20]

symptoms = get_symptoms(mapped_disease)

# -----
# SEVERITY
# -----

if confidence >= 95:
    severity = "High"

elif confidence >= 80:
    severity = "Moderate"

else:
    severity = "Low"

```

```
# -----  
# RECOMMENDATIONS  
# -----  
  
if disease.lower() == "covid":  
    causes = "SARS-CoV-2 Viral Infection"  
  
    tests = [  
        "RT-PCR Test",  
        "Rapid Antigen Test",  
        "Chest CT Scan"  
    ]  
  
    treatments = [  
        "Rest",  
        "Hydration",  
        "Antiviral Medication",  
        "Medical Monitoring"  
    ]  
  
    prevention = [  
        "Vaccination",  
        "Mask Usage",  
        "Hand Hygiene",  
        "Social Distancing"  
    ]  
  
elif disease.lower() == "viral pneumonia":  
    causes = "Viral Infection Affecting Lungs"  
  
    tests = [  
        "Chest X-Ray",  
        "Blood Test",  
        "CT Scan"  
    ]  
  
    treatments = [  
        "Antiviral Medication",  
        "Oxygen Support",  
        "Rest"  
    ]
```

```

        prevention = [
            "Vaccination",
            "Healthy Lifestyle",
            "Avoid Smoking"
        ]
    else:

        causes = "Healthy Chest X-Ray"

        tests = [
            "Routine Health Checkup"
        ]

        treatments = [
            "No Treatment Required"
        ]

        prevention = [
            "Regular Exercise",
            "Balanced Diet",
            "Adequate Sleep"
        ]

# -----
# PRINT REPORT
# -----

print("\n")
print("="*70)
print("                AI GENERATED MEDICAL REPORT")
print("="*70)

print(f"\nPredicted Disease : {disease}")
print(f"Confidence Score   : {confidence:.2f}%")
print(f"Severity Level      : {severity}")

print("\nPossible Cause:")
print(causes)

print("\nLikely Symptoms:")

for symptom in symptoms:
    print("•", symptom)

```

```
print("\nRecommended Tests:")

for item in tests:
    print("•", item)

print("\nSuggested Treatment:")

for item in treatments:
    print("•", item)

print("\nPrevention Tips:")

for item in prevention:
    print("•", item)

print("\nDoctor Recommendation:")
print(
    "Consult a certified healthcare professional "
    "for proper diagnosis and treatment."
)

print("\nDISCLAIMER:")
print(
    "This report is generated using Artificial "
    "Intelligence and should not be considered "
    "a final medical diagnosis."
)

print("="*70)

# -----
# SAVE PDF REPORT
# -----

pdf = SimpleDocTemplate(
    "AI_Medical_Report.pdf"
)

styles = getSampleStyleSheet()

elements = []

elements.append(
    Paragraph(
        "AI Generated Medical Report",
```

```
        styles['Title']
    )
)
elements.append(Spacer(1,12))

elements.append(
    Paragraph(
        f"<b>Predicted Disease:</b> {disease}",
        styles['Normal']
    )
)

elements.append(
    Paragraph(
        f"<b>Confidence Score:</b> {confidence:.2f}%",
        styles['Normal']
    )
)

elements.append(
    Paragraph(
        f"<b>Severity Level:</b> {severity}",
        styles['Normal']
    )
)

elements.append(Spacer(1,12))

elements.append(
    Paragraph(
        "<b>Possible Cause</b>",
        styles['Heading2']
    )
)

elements.append(
    Paragraph(
        causes,
        styles['Normal']
    )
)

elements.append(Spacer(1,12))
```

```
elements.append(
    Paragraph(
        "<b>Likely Symptoms</b>",
        styles['Heading2']
    )
)

for symptom in symptoms:

    elements.append(
        Paragraph(
            symptom,
            styles['Normal']
        )
    )

elements.append(Spacer(1,12))

elements.append(
    Paragraph(
        "<b>Recommended Tests</b>",
        styles['Heading2']
    )
)

for item in tests:

    elements.append(
        Paragraph(
            item,
            styles['Normal']
        )
    )

elements.append(Spacer(1,12))

elements.append(
    Paragraph(
        "<b>Suggested Treatment</b>",
        styles['Heading2']
    )
)

for item in treatments:
```



```
        elements.append(
            Paragraph(
                item,
                styles['Normal']
            )
        )

elements.append(Spacer(1,12))

elements.append(
    Paragraph(
        "<b>Prevention Tips</b>",
        styles['Heading2']
    )
)

for item in prevention:

    elements.append(
        Paragraph(
            item,
            styles['Normal']
        )
    )

elements.append(PageBreak())

elements.append(
    Paragraph(
        "DISCLAIMER",
        styles['Heading1']
    )
)

elements.append(
    Paragraph(
        "This report is generated by an AI model "
        "and should only be used for educational "
        "and research purposes. Consult a medical "
        "professional before making healthcare "
        "decisions.",
        styles['Normal']
    )
)
```

Requirement already satisfied: reportlab in /usr/local/lib/python3.12/dist-packages (4.5.1)
Requirement already satisfied: pillow>=9.0.0 in /usr/local/lib/python3.12/dist-packages (from reportlab)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from reportlab)

=====

AI GENERATED MEDICAL REPORT

=====

Predicted Disease : Uncertain
Confidence Score : 51.71%
Severity Level : Low

Possible Cause:
Healthy Chest X-Ray

Likely Symptoms:
• No symptom information available

Recommended Tests:
• Routine Health Checkup

Suggested Treatment:
• No Treatment Required

Prevention Tips:
• Regular Exercise
• Balanced Diet
• Adequate Sleep